

Kişisel LiteLLM Geliştirmelerini Entegre Etme

Bu üç geliştirmeyi, sizinkiyle aynı sürüm olan başka bir LiteLLM kurulumuna gerçek kod değişiklikleri (git patch) olarak uygulama rehberi — yeniden yazmaya gerek yok, olduğu gibi uygulanır.

Neyi Entegre Ediyoruz

1. Universal Request & Response Loglama (Logs sekmesi)

- prettyMessagesTypes.ts
- prettyMessagesUtils.ts
- OutputCard.tsx

Logs > bir isteğe tıklayınca açılan 'Request & Response' paneli artık sadece chat modelleri için değil, text-completion / image-generation / embedding modelleri için de gerçek input/output gösteriyor (öncesinde chat dışında hep boştu).

2. Metrics Sekmesi (tüm grafikler + hesaplamalar)

- perf-stats/page.tsx (yeni sayfa)
- leftnav.tsx (sol menüye 'Metrics' linki)
- migratedPages.ts (route kaydı)
- + scripts/dashboard.py (ayrı Python script, patch'in dışında)

Sol menüde yeni bir 'Metrics' sekmesi -- model bazlı/tüm modeller birleşik cevap süresi grafikleri, günlük/saatlik trend, model başına detaylı istatistik (P95 gecikme, cache hit oranı vb).

3. Playground'da Chat Dışı Modlar (/v1/completions)

- mode_endpoint_mapping.tsx
- chatConstants.ts
- ChatUI.tsx
- CodeSnippets.tsx
- text_completion.tsx (yeni dosya)

Playground'un Endpoint Type listesine '/v1/completions' eklendi -- chat template'i olmayan base/completion modelleri (örn. ham bir GPT-2) artık Playground'dan test edilebiliyor (öncesinde sadece /v1/chat/completions vardı, bu tür modeller hep 400 hatası veriyordu).

BÖLÜM A — ÖN KOŞULLAR

1. LiteLLM sürümünüzü doğrulayın

Patch, LiteLLM Admin UI kaynak kodunun (ui/litellm-dashboard) v1.97.0 sürümüne göre hazırlandı. Farklı bir sürümdeyseniz patch'in bazı kısımları uyuşmayabilir (dosyalar zamanla değişiyor).

```
curl -s http://localhost:4000/health/liveliness | grep -o '"litellm_version":"[^"]*"'
# ya da Admin UI'de sol ust kosedede: v1.97.0 gibi bir etiket gorursunuz
```

NOT: Sürümünüz farklıysa endişelenmeyin -- Bölüm B'deki 'sparse checkout' adımı kendi sürümünüzün tag'ini kullanırsanız patch büyük ihtimalle yine uygulanır (değişen dosyalar nadiren bu kadar sık değişir). Uygulanmazsa 'git apply --reject' ile kısmi uygulayıp kalanları elle taşıyabilirsiniz.

2. Elinizde olması gerekenler

Bu rehberle birlikte gelen 2 dosya:

. Teslim edilen dosyalar

- personal-patches.diff (11 dosyalık git patch -- Bölüm A/B'nin konusu)
- dashboard.py (Metrics sekmesinin arka planda çalıştırdığı script)

BÖLÜM B — UI KAYNAK KODUNU HAZIRLAMA VE PATCH'İ UYGULAMA

3. LiteLLM Admin UI kaynak kodunu (aynı sürüm) çekin

Admin UI, pip/docker paketinin içine derlenmiş statik dosyalar olarak gömülü -- kaynağı GitHub'dan sürümünüze eşit bir tag ile ayrı çekiyoruz. Sparse checkout kullanıyoruz çünkü tüm LiteLLM reposu çok büyük, sadece UI klasörü lazım.

```
git clone --depth 1 --branch v1.97.0 --filter=blob:none --sparse \
  https://github.com/BerriAI/litellm.git litellm-ui-fork
cd litellm-ui-fork
git sparse-checkout set ui/litellm-dashboard

# v1.97.0 yerine kendi surumunuzun tag'ini yazın
```

4. Patch'i uygulayın

personal-patches.diff dosyasını litellm-ui-fork klasörünün köküne kopyalayıp uygulayın:

```
cd litellm-ui-fork
git apply --check personal-patches.diff # once sadece dogrulama (dosya degismez)
git apply personal-patches.diff        # gercekten uygula
```

NOT: 'git apply --check' hatasız biterse güvenle uygulayabilirsiniz. Hata verirse (genelde sürüm farkından), hangi dosyada çakıştığını mesajdan görüp o dosyayı elle karşılaştırarak taşıyabilirsiniz -- patch içeriği küçük ve okunabilir (toplam ~110 satır ekleme).

5. UI'yi yeniden derleyin

Next.js statik export'u Docker içinde (host'ta Node.js kurmadan) üretiyoruz -- 'out/' klasörü çıktı olarak host'a kopyalanır:

```
docker run --rm \
-v "$(pwd)/ui/litellm-dashboard:/src:ro" \
-v "$(pwd)/ui/litellm-dashboard/out:/host_out" \
node:24 sh -c "
  cp -r /src /app && rm -rf /app/node_modules /app/out &&
  cd /app && npm ci --no-audit --no-fund && npm run build &&
  cp -r /app/out/. /host_out/
"
```

NOT: Bu adım birkaç dakika sürer (npm ci + next build). Bittiğinde litellm-ui-fork/ui/litellm-dashboard/out/ klasöründe hazır statik dosyalar olur.

BÖLÜM C — LITELLM PROXY'YE BAĞLAMA

6. Derlenen UI'yi konteynere bağlayın

docker-compose.yml'deki litellm servisine, derlediğiniz out/ klasörünü konteynerin içindeki gömülü UI yolunun üzerine salt-okunur (read-only) olarak bağlayan bir volume ekleyin -- LiteLLM paketinin kendisi hiç değişmez, sadece hangi statik dosyaları servis ettiği değişir:

```
services:
  litellm:
    # ...mevcut ayarlarınız aynen kalır...
    volumes:
      # NOT: asagidaki satir gercekte TEK satir (kaynak:hedef:ro), burada
      # sadece okunabilirlik icin \ ile bolundu -- kendi dosyanizda birlestirin:
      - ./litellm-ui-fork/ui/litellm-dashboard/out:\
        /app/.venv/lib/python3.13/site-packages/litellm/proxy/_experimental/out:ro
```

NOT: Yol içindeki 'python3.13' kısmı, LiteLLM image'ının kullandığı Python sürümüne göre değişebilir -- konteynerin içinde 'find / -path */proxy/_experimental/out -type d' ile gerçek yolu doğrulayabilirsiniz.

7. Universal loglama için gerekli ayarlar (Özellik 1)

Patch kendisi loglamayı GÖSTERİR, ama gösterecek veri DB'de olmalı -- LiteLLM varsayılan olarak prompt/response metnini kaydetmez. Aynı litellm servisine şu ortam değişkenlerini ekleyin:

```
environment:
  STORE_PROMPTS_IN_SPEND_LOGS: "True"          # prompt+cevabi DB'ye yaz
  MAX_STRING_LENGTH_PROMPT_IN_DB: "2000000"    # uzun promptların kirpilmesini önler
```

NOT: KVKK/GDPR notu: STORE_PROMPTS_IN_SPEND_LOGS açıkken kullanıcı promptları veritabanında kalıcı olarak saklanır -- üretim ortamında açmadan önce bunu değerlendirin.

8. Metrics script'ini çalıştırın (Özellik 2)

dashboard.py, Postgres'e docker exec ile sorgu atan bağımsız bir Python scripti -- LiteLLM paketinin bir parçası değil, ayrı çalışır. Dosyanın başındaki 3 sabiti kendi ortamınıza göre düzenleyin:

```
PORT = 8093
PG_CONTAINER = "litellm-postgres"  # kendi Postgres konteyner adınız
PG_USER = "litellm"                # kendi DB kullanici adiniz
PG_DB = "litellm"                  # kendi DB adiniz
```

Sonra çalıştırın (LiteLLM proxy'nizle aynı makinede, Docker CLI'ye erişimi olan bir yerden):

```
python3 scripts/dashboard.py &  
# http://localhost:8093 üzerinde ayaga kalkar
```

NOT: perf-stats/page.tsx dosyasında (patch'te gelen) METRICS_DASHBOARD_URL sabiti 'http://localhost:8093' olarak gömülü -- farklı bir port/host kullanacaksanız 6. adımdan önce bu dosyada da güncelleyip yeniden derleyin.

9. Proxy'yi yeniden başlatın

```
docker compose up -d litellm --force-recreate
```

Doğrulama Kontrol Listesi

Özellik	Nasıl test edilir
1. Universal loglama	Logs sekmesi -> bir completion/image/embedding isteğine tıklayın -> 'Request & Response' panelinde artık gerçek input/output görünmeli (önceden boştu)
2. Metrics	Sol menüde 'Metrics' linkine tıklayın -> grafikler gerçek verilerle yüklenmeli
3. Playground /v1/completions	Playground -> Endpoint Type -> '/v1/completions' seçeneği görünmeli; bir base/completion modeli seçip mesaj gönderin, hata almamalısınız

Sık Karşılaşılan Sorunlar

Belirti	Muhtemel sebep
git apply hata veriyor	LiteLLM sürümü v1.97.0'dan farklı ve o dosyalar değişmiş -- 3. adımda kendi sürümünüzün tag'ini kullandığınızdan emin olun
Metrics sekmesi boş/hata veriyor	dashboard.py çalışmıyor olabilir, ya da PG_CONTAINER/PG_USER/PG_DB yanlış
Logs'ta hâlâ input/output boş	STORE_PROMPTS_IN_SPEND_LOGS ayarlanmamış, ya da eski (patch öncesi) isteklere bakıyorsunuz -- yeni bir istek atıp tekrar deneyin
UI hiç değişmemiş görünüyor	6. adımdaki volume yolu yanlış olabilir -- konteyner içinde gerçek yolu doğrulayın